

Lecture 3: Introduction to Data - Part 2

LSE ME314: Introduction to Data Science and Machine Learning (<https://github.com/me314-lse>)

2026-07-15

Daniel de Kadt

Where Do Data Come From?

Data Sources

Typically we think of data as coming from a 'source:'

- API
- Database
- Users
- Surveys
- Experiments

Data Sources

Typically we think of data as coming from a 'source:'

- API
- Database
- Users
- Surveys
- Experiments

But where do data *really* come from?

Adventure Awaits!



Source: Wizards of the Coast, [DND Beginners Guide](#)

Adventure Awaits!

Gather your party, and venture forth to find out...

```
# View our characters:
```

```
head(dnd_characters)
```

	Name	Class	Race	Gender	Strength	Constitution			
1	Thinos	Barbarian	Tiefling	Female	9	14			
2	Theax	Barbarian	Dwarf	Male	8	13			
3	Thuldor	Fighter	Human	Male	10	14			
4	Nyax	Stormborn	Barbarian	Halfling	Female	10	13		
5	Droador	Rogue	Elf	Male	15	12			
6	Korinar	Barbarian	Dragonborn	Non-binary	15	14			
	Dexterity	Wisdom	Charisma	Intelligence	Hitpoints	Attack	AC	Die	
1	15	13	12	14	14	1	12	12	
2	9	10	11	8	13	1	9	12	
3	14	10	7	10	12	2	12	8	
4	10	13	13	9	13	2	10	12	
5	16	14	11	9	9	4	13	6	
6	13	14	18	8	14	4	11	12	

Probability

What is a Probability?

Random variable: A variable that is subject to probability.

What is a Probability?

Random variable: A variable that is subject to probability.

Sample space: The set of all possible outcomes.

Event: A subset of the sample space - something that could occur.

Outcome: A realization of the random variable - what happens.

What is a Probability?

Random variable: A variable that is subject to probability.

Sample space: The set of all possible outcomes.

Event: A subset of the sample space - something that could occur.

Outcome: A realization of the random variable - what happens.

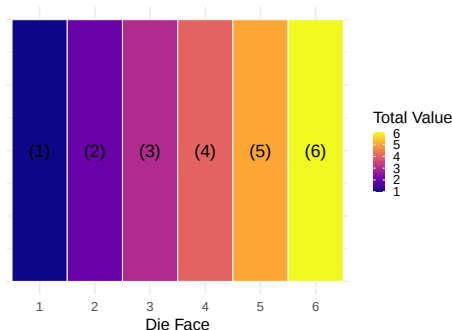
Probability: The chance that the outcome will be a particular event in the sample space.

Sample Space

A **sample space** is denoted by Ω :

```
# Create sample space of a single roll of a D6:  
df_single <- data.frame(  
  rollvalue = factor(1:6),  
  row = 1  
)
```

Here $\Omega = \{1, 2, 3, 4, 5, 6\}$ and each value is an event. Visually:



Events, Probabilities, and the Probability Axioms

Consider one event A in the sample space Ω .

This could be one outcome of a roll, e.g. A is defined as $roll = 3$

The probability of A , denoted $P(A)$ (or $P(roll = 3)$)

Events, Probabilities, and the Probability Axioms

Consider one event A in the sample space Ω .

This could be one outcome of a roll, e.g. A is defined as $roll = 3$

The probability of A , denoted $P(A)$ (or $P(roll = 3)$)

Ω and all events therein (including A) are constrained by three axioms (Kolmogorov):

1. **Non-negativity**: $P(A) \geq 0$.
2. **Normalization**: $P(\Omega) = 1$.
3. **Additivity**: If $A_1, A_2, \dots, A_n$ are mutually exclusive events, then $P(A_1 \cup A_2 \cup \dots \cup A_n) = P(A_1) + P(A_2) + \dots + P(A_n)$.

Joint Sample Spaces

What if we have two probabilistic processes?

Joint Sample Spaces

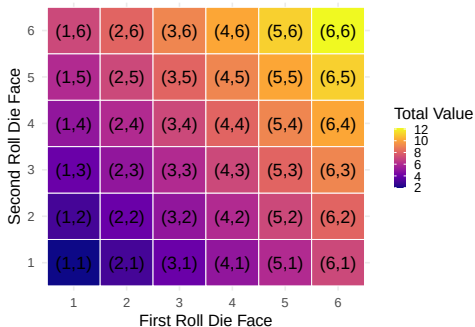
What if we have two probabilistic processes?

```
# Create sample space for two rolls of a D6:  
df_double <- expand.grid(  
  roll1value = 1:6,  
  roll2value = 1:6  
)
```

Joint Sample Spaces

What if we have two probabilistic processes?

```
# Create sample space for two rolls of a D6:  
df_double <- expand.grid(  
  roll1value = 1:6,  
  roll2value = 1:6  
)
```



Joint Sample Spaces

With multiple probabilistic processes occurring together, we are describing a joint sample space.

We can reason about $P(\text{roll1} = x1)$ or $P(\text{roll2} = x2)$ or:

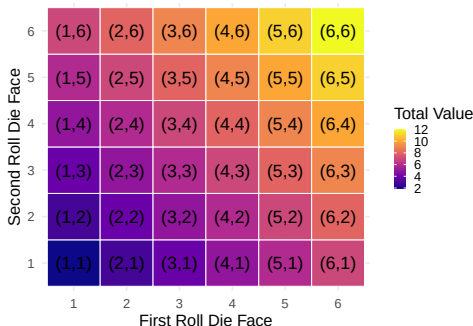
- Union (either or): $P(\text{roll1} = x1 \cup \text{roll2} = x2)$
- Intersection (both): $P(\text{roll1} = x1 \cap \text{roll2} = x2)$

Joint Sample Spaces

With multiple probabilistic processes occurring together, we are describing a joint sample space.

We can reason about $P(\text{roll1} = x1)$ or $P(\text{roll2} = x2)$ or:

- Union (either or): $P(\text{roll1} = x1 \cup \text{roll2} = x2)$
- Intersection (both): $P(\text{roll1} = x1 \cap \text{roll2} = x2)$



Probability Distributions

A random variable is drawn from a probability distribution.

Probability distributions are defined by:

1. **Support:** The set of values that the variable can take
2. **Function:**
 - **Probability Mass Function (PMF):** For discrete variables, the PMF gives the exact probability of each possible outcome
 - **Probability Density Function (PDF):** For continuous variables, gives a relative likelihood of possible outcomes in an interval (not a probability!)
 - **Parameters:** Values that determine the shape and characteristics of the PMF or PDF
3. **Cumulative Distribution Function (CDF):** The CDF gives the probability that the variable is less than or equal to a given value ($P(X \leq x)$).

Probability Distributions: Uniform (Discrete)

For random variable X :

We denote $X \sim \mathcal{U}(a, b)$

Probability Distributions: Uniform (Discrete)

For random variable X :

We denote $X \sim \mathcal{U}(a, b)$

This means “the random variable X follows a uniform distribution between a and b ”

Or “the random variable X is drawn from a uniform distribution with support $[a, b]$ ”

(a and b are the lower and upper boundaries of support)

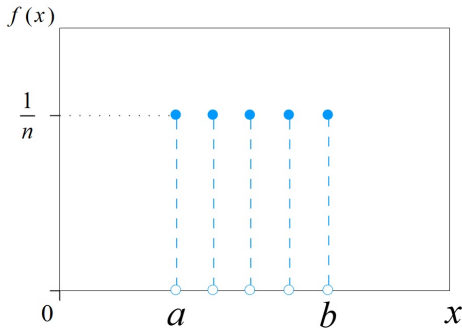
Probability Distributions: Uniform (Discrete)

For random variable X : $X \sim \mathcal{U}(a, b)$:

Probability Distributions: Uniform (Discrete)

For random variable X : $X \sim \mathcal{U}(a, b)$:

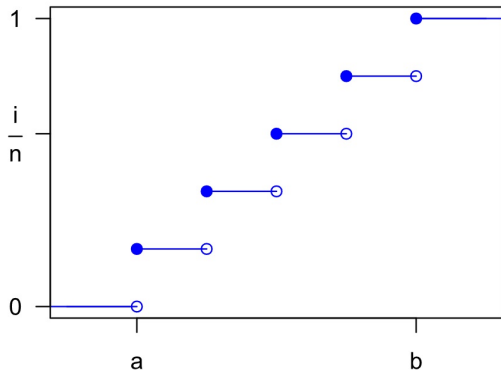
The PMF is written as: $P(X = x) = \frac{1}{n}$ where n is the number of unique values in the support.



Source: Wikipedia, [Discrete Uniform Distribution](#)

Probability Distributions: Uniform (Discrete)

The CDF looks like this:



Source: Wikipedia, [Discrete Uniform Distribution](#)

Probability Distributions: Binomial

The binomial distribution models the outcome of a series of independent Bernoulli trials (coin flips)

Each trial has two possible outcomes: heads (with probability p) or tails (with probability $1 - p$)

Probability Distributions: Binomial

The binomial distribution models the outcome of a series of independent Bernoulli trials (coin flips)

Each trial has two possible outcomes: heads (with probability p) or tails (with probability $1 - p$)

The total number of trials, n , is fixed, and the trials are independent of each other (what you get in one coin flip has no effect on any other coin flip)

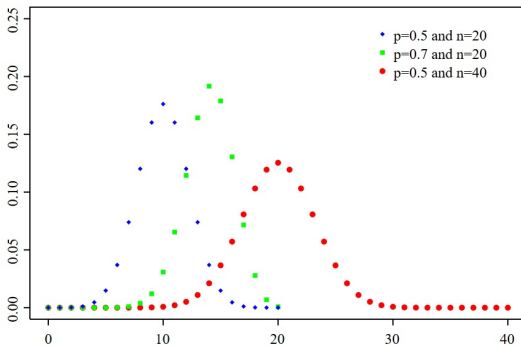
For random variable X : $X \sim \mathcal{B}(n, p)$.

Probability Distributions: Binomial

The PMF is given by:

$$P(X = k) = \binom{n}{k} p^k (1 - p)^{n-k}$$

where $\binom{n}{k}$ is the binomial coefficient, which counts the number of ways to choose k successes in n trials.

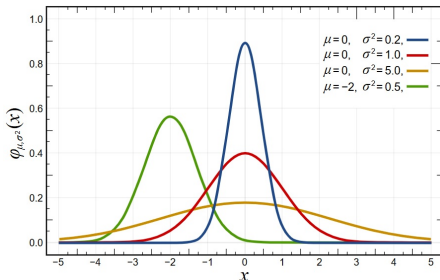


Probability Distributions: Normal or Gaussian

A normal distribution is a continuous probability distribution.

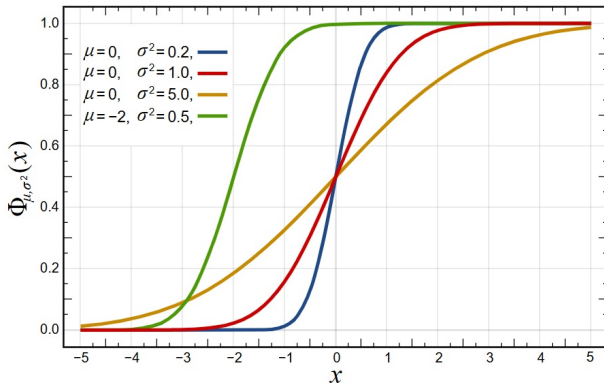
For random variable X : $X \sim \mathcal{N}(\mu, \sigma^2)$, where μ is the mean, and σ^2 is the variance.

The PDF is given by $\frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$:



Probability Distributions: Normal or Gaussian

And the CDF of the normal:



Source: Wikipedia, [Normal Distribution](#)

Probability Distributions in R

Let's sample from a normal distribution in R using `rnorm()`, setting the mean and standard deviation:

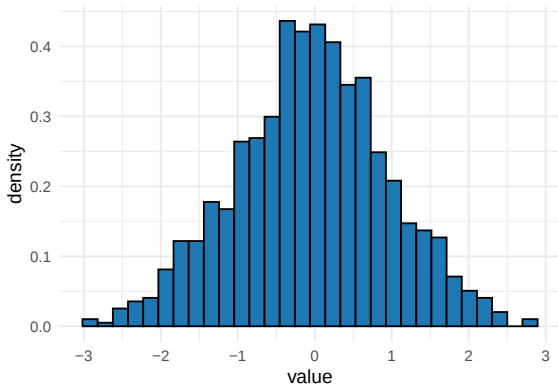
```
normal_sample <- data.frame(value = rnorm(1000, mean = 0, sd = 1))

normal_hist <- normal_sample %>%
  ggplot(aes(x = value)) +
  geom_histogram(aes(y = after_stat(density)), bins = 30,
                 fill = "#1f77b4", color = "black") +
  theme_minimal(base_size = 24)
```

What's going on here? Each value in `normal_sample$value` is a single draw from a normal distribution with mean 0 and standard deviation 1.

We then plot a histogram of the drawn values.

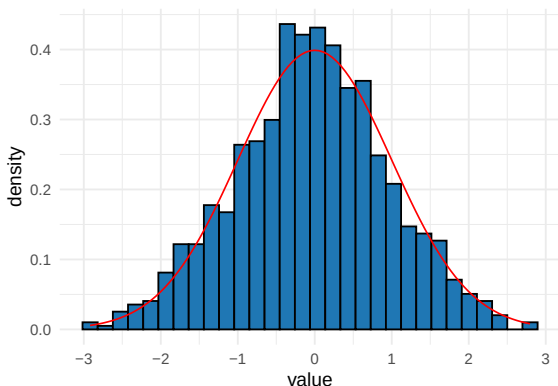
Probability Distributions in R



Probability Distributions in R

Let's overlay the actual probability density function for the normal distribution ($\mu = 0, \sigma^2 = 1$):

```
normal_hist + # this is the previous ggplot2 object, now we add a layer
  stat_function(fun = dnorm, args = list(mean = 0, sd = 1),
               color = "red", linewidth = 1)
```



Probability Distributions in R

You can do similar things for other distributions:

- `runif()`: randomly draw from a uniform distribution
- `dunif()`: density for a uniform distribution
- `rbinom()`: randomly draw from a binomial distribution
- `dbinom()`: density for a binomial distribution
- `rpois()`: randomly draw from a Poisson distribution
- etc.

Each function takes different arguments depending on the parameters required.

Some Theory: Expected Values

A random variable is said to have an **expected value** of $\mathbb{E}(X)$, which is the average value of the random variable over very many trials.

Intuitively you can think of this as the mean of all the possible values X could take, weighted by the probability of each value being realized.

Some Theory: Expected Values

A random variable is said to have an **expected value** of $\mathbb{E}(X)$, which is the average value of the random variable over very many trials.

Intuitively you can think of this as the mean of all the possible values X could take, weighted by the probability of each value being realized.

The expected value of:

- a uniform distribution: $\mathbb{E}(X) = \frac{a+b}{2}$.
- a normal distribution: $\mathbb{E}(X) = \mu$.
- a binomial distribution: $\mathbb{E}(X) = n \cdot p$.

Some Theory: Expected Values

Let's demonstrate this in R:

Some Theory: Expected Values

Let's demonstrate this in R:

```
# Sample 1000 times from a uniform dist with a = 0, b = 10
draw <- runif(1000, min = 0, max = 10)
# Calculate the average (should be close to  $(a+b)/2$ )
mean(draw)
```

```
[1] 4.88331
```

Some Theory: Expected Values

Let's demonstrate this in R:

```
# Sample 1000 times from a uniform dist with a = 0, b = 10
draw <- runif(1000, min = 0, max = 10)
# Calculate the average (should be close to  $(a+b)/2$ )
mean(draw)
```

```
[1] 4.88331
```

```
# Sample 1000 times from a binomial dist with n = 1, p = 0.5
draw <- rbinom(1000, size = 1, prob = 0.5)
# Calculate the average (should be close to  $n \cdot p$ )
mean(draw)
```

```
[1] 0.48
```

Some Theory: Expected Values

Let's demonstrate this in R:

```
# Sample 1000 times from a uniform dist with a = 0, b = 10
draw <- runif(1000, min = 0, max = 10)
# Calculate the average (should be close to  $(a+b)/2$ )
mean(draw)
```

```
[1] 4.88331
```

```
# Sample 1000 times from a binomial dist with n = 1, p = 0.5
draw <- rbinom(1000, size = 1, prob = 0.5)
# Calculate the average (should be close to  $n \cdot p$ )
mean(draw)
```

```
[1] 0.48
```

```
# Sample 1000 times from a normal dist with mean = 5, sd = 5
draw <- rnorm(1000, mean = 5, sd = 5)
# Calculate the average (should be close to  $\text{mean}$ )
mean(draw)
```

```
[1] 5.059436
```

Some Theory: Law of Large Numbers

What happened is an example of the **Law of Large Numbers** (LLN).

As the number of i.i.d* observations in our sample increases, the sample mean converges to the true expected value:

$$\bar{X}_n \longrightarrow \mathbb{E}(X) \text{ as } n \rightarrow \infty$$

where $\bar{X}_n$ is the sample mean of n realisations of the random variable X .

*i.i.d = independent and identically distributed

Some Theory: Central Limit Theorem

Let's connect the LLN to a deeper result: the **Central Limit Theorem** (CLT).

Over K repeated samples each of size n , the *distribution* of the sample mean $\bar{X}_k$ approaches a normal distribution as $K \rightarrow \infty$.

This is true regardless of the underlying probability distribution from which X is drawn.

This will be the backbone of asymptotic statistical inference (tomorrow).

Some Theory: Central Limit Theorem

Let's do the following simulation for K repeated samples:

1. Pick a probability distribution to generate X
2. K times:
 - 2.1 Draw one sample of size n : $X_{1k}, X_{2k}, \dots, X_{nk}$
 - 2.2 Calculate and store the sample mean $\bar{X}_k$
3. Plot the distribution of all K sample means.

Theory in Practice

We'll do the simulation with our favourite dice, the D6:

```
d6_sampler <- function(K, n) {  
  
  sample_mean <- numeric(K)  
  
  for (k in 1:K) {  
    # Roll a D6 `n` times to create an i.i.d sample  
    rolls <- sample(1:6, n, replace = TRUE)  
    # Calculate the sample mean  
    sample_mean[k] <- mean(rolls)  
  }  
  
  return(sample_mean)  
}
```

Theory in Practice

We'll do the simulation with our favourite dice, the D6:

```
d6_sampler <- function(K, n) {  
  
  sample_mean <- numeric(K)  
  
  for (k in 1:K) {  
    # Roll a D6 `n` times to create an i.i.d sample  
    rolls <- sample(1:6, n, replace = TRUE)  
    # Calculate the sample mean  
    sample_mean[k] <- mean(rolls)  
  }  
  
  return(sample_mean)  
}
```

Start with just a single draw of 100 rolls (LLN at work):

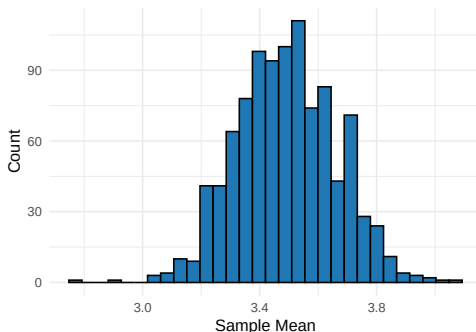
```
d6_sampler(K = 1, n = 100)
```

```
[1] 3.29
```

Theory in Practice

Now draw 1000 samples each of size 100 (CLT at work):

```
many_d6_samples <- d6_sampler(K = 1000, n = 100)
many_d6_samples %>%
  as.data.frame() %>%
  ggplot(aes(x = many_d6_samples)) +
  geom_histogram(bins = 30, fill = "#1f77b4", color = "black") +
  labs(x = "Sample Mean", y = "Count") +
  theme_minimal(base_size = 24)
```



Into Battle

A Probabilistic Duel

Let's pull two of our characters and settle a score with dice. A rage filled Barbarian and a battle-hardened Fighter going head to head.



A Probabilistic Duel

Every character has a number of combat stats – an attack bonus (from Strength), an armour class (AC, from Dexterity), and a damage die (set by class):

```
# Pull one Barbarian and one Fighter from our characters
barbarian <- dnd_characters %>% filter(Class == "Barbarian") %>% slice(1)
fighter   <- dnd_characters %>% filter(Class == "Fighter")   %>% slice(1)

bind_rows(barbarian, fighter) %>%
  select(Name, Class, Hitpoints, Attack, AC, Die)
```

	Name	Class	Hitpoints	Attack	AC	Die
1	Thinos	Barbarian	14	1	12	12
2	Thuldor	Fighter	12	2	12	8

Every Swing is a Coin Flip

Attacks land if $d20 + \text{Attack}$ reaches the target's Armour Class.

You can think of this as a Bernoulli trial, just like our coin flips.

Because all 20 faces of the d20 are equally likely, we can compute each trial's success probability exactly:

```
# Probability an attack lands: P(d20 + Attack >= target's AC)
hit_prob <- function(attacker, defender) {
  mean(1:20 + attacker$Attack >= defender$AC)
}
```

```
hit_prob(barbarian, fighter) # P(Barbarian hits Fighter)
```

```
[1] 0.5
```

```
hit_prob(fighter, barbarian) # P(Fighter hits Barbarian)
```

```
[1] 0.55
```

Settling a Score

To simulate a duel between our two characters, we flip a coin for who swings first, then trade blows until someone drops to 0 hitpoints.

A while loop keeps the duel going as long as both combatants are standing:

```
simulate_duel <- function(a, b) {  
  # Flip a coin for who swings first (shuffle the pair)  
  fighters <- sample(list(a, b))  
  a <- fighters[[1]]  
  b <- fighters[[2]]  
  hp_a <- a$Hitpoints  
  hp_b <- b$Hitpoints  
  # Trade blows while both are still standing  
  while (hp_a > 0 && hp_b > 0) {  
    if (sample(1:20, 1) + a$Attack >= b$AC) hp_b <- hp_b - sample(1:a$Die, 1)  
    if (hp_b <= 0) break # b falls before it can swing back  
    if (sample(1:20, 1) + b$Attack >= a$AC) hp_a <- hp_a - sample(1:b$Die, 1)  
  }  
  # The survivor wins  
  if (hp_a > 0) a$Name else b$Name  
}  
set.seed(1)  
simulate_duel(barbarian, fighter) # Who wins this time?
```

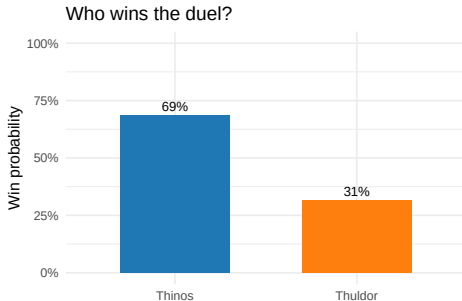
```
[1] "Thinots"
```

One Duel Isn't Enough

A single duel is one draw from a random process. Let's run the duel 10,000 times and average:

```
set.seed(1)
duels <- replicate(10000, simulate_duel(barbarian, fighter))
prop.table(table(duels))
```

```
duels
Thinos Thuldor
0.6852  0.3148
```



So, Where Do Data *Really* Come From?

Data Generating Processes

All data we encounter come from underlying **Data Generating Processes** (DGPs).

We (usually) cannot see these processes, and typically only observe *one* draw (sample).

Data Generating Processes

All data we encounter come from underlying **Data Generating Processes** (DGPs).

We (usually) cannot see these processes, and typically only observe *one* draw (sample).

We often want evaluate Y given some set of features $X_1, X_2, \dots, X_n$ – that is, we want to **compare** the value of Y we expect to see given different values of our features.

To do this we often build a model of Y – this is us making some *assumptions* (often parametric) about the DGP that generated Y .

These may or may not be good assumptions.

Data Generating Processes

Consider this **Conditional Expectation Function** (CEF):

$$\mathbb{E}(Y \mid X_1, X_2, \dots, X_n) = f(X_1, X_2, \dots, X_n)$$

where $f(\cdot)$ is some function that describes the relationship between the features and the outcome.

Evaluated for any particular $X_1 = x_1, X_2 = x_2, \dots, X_n = x_n$, this returns the expected value of Y .

Data Generating Processes

Consider this **Conditional Expectation Function** (CEF):

$$\mathbb{E}(Y \mid X_1, X_2, \dots, X_n) = f(X_1, X_2, \dots, X_n)$$

where $f(\cdot)$ is some function that describes the relationship between the features and the outcome.

Evaluated for any particular $X_1 = x_1, X_2 = x_2, \dots, X_n = x_n$, this returns the expected value of Y .

What should $f(\cdot)$ look like?

Data Generating Processes

Consider this **Conditional Expectation Function** (CEF):

$$\mathbb{E}(Y \mid X_1, X_2, \dots, X_n) = f(X_1, X_2, \dots, X_n)$$

where $f(\cdot)$ is some function that describes the relationship between the features and the outcome.

Evaluated for any particular $X_1 = x_1, X_2 = x_2, \dots, X_n = x_n$, this returns the expected value of Y .

What should $f(\cdot)$ look like? Well, that is the million dollar question.

Data Generating Processes

We will think about DGPs as a function with different possible components:

- **Deterministic:**
 - Typically 'fixed' or 'given'
- **Probabilistic:**
 - Product of a probabilistic process

These probabilistic components can themselves be of different types:

- *Systematic:*
 - Product of other variables, of which at least one is probabilistic
- *Stochastic:*
 - Random processes independent of other features

Data Generating Processes

We will think about DGPs as a function with different possible components:

- **Deterministic:**

- Typically 'fixed' or 'given'

- **Probabilistic:**

- Product of a probabilistic process

These probabilistic components can themselves be of different types:

- *Systematic:*

- Product of other variables, of which at least one is probabilistic

- *Stochastic:*

- Random processes independent of other features

Deep philosophical point: Is anything ever actually random?

(Probably not)

Data Generating Processes

Let's return to the CEF of Y :

- Assume each of $X_1, X_2, \dots, X_n$ is drawn from a probability distribution
- If Y is determined by these variables, then Y is a probabilistic function of these underlying distributions

This can become extremely complicated – we could allow $X_1, X_2, \dots, X_n$ to be correlated or jointly drawn from a multivariate probability distribution.

Data Generating Processes

Let's return to the CEF of Y :

- Assume each of $X_1, X_2, \dots, X_n$ is drawn from a probability distribution
- If Y is determined by these variables, then Y is a probabilistic function of these underlying distributions

This can become extremely complicated – we could allow $X_1, X_2, \dots, X_n$ to be correlated or jointly drawn from a multivariate probability distribution.

Key insight: One DGP could produce very many unique datasets. But we only observe one.

DGPs in Action

Let's return to our D&D characters.

What features did our data have:

- **Deterministic:**

- Name
- Class
- Race

- **Probabilistic:**

- *Stochastic:*

- Strength, Constitution, Dexterity, Intelligence, Wisdom, Charisma

- *Systematic:*

- Hitpoints (based on Constitution)
- Attack (based on Strength) and AC (based on Dexterity)

Our D&D characters are a sample from a DGP!

DGPs in Action

The stochastic components (the 'stats' or attributes):

DGPs in Action

The stochastic components (the 'stats' or attributes):

```
roll_4d6 <- function() {  
  rolls <- sample(1:6, 4, replace = TRUE)  
  sum(sort(rolls, decreasing = TRUE)[1:3])  
}
```

DGPs in Action

The stochastic components (the 'stats' or attributes):

```
roll_4d6 <- function() {  
  rolls <- sample(1:6, 4, replace = TRUE)  
  sum(sort(rolls, decreasing = TRUE)[1:3])  
}
```

The systematic components are derived from the above (and a few other bits):

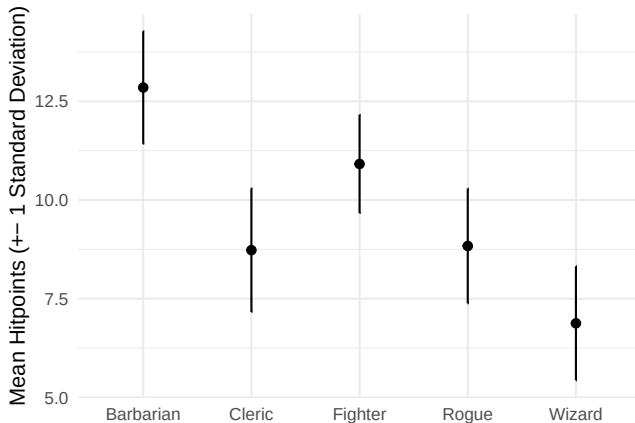
```
# Note: The below is pseudo-code!  
# Assigning class (this was actually random!)  
class <- sample(c("Fighter", "Wizard", "Rogue", "Cleric", "Barbarian"), 1)  
# A function to calculate 'modifiers':  
modifier <- function(score) floor((score - 10) / 2)  
# Calculating the constitution modifier from a stats object:  
con_mod <- modifier(stats[["Constitution"]])  
# A dictionary of hit dice by class  
hit_dice <- c(Fighter = 10, Wizard = 6, Rogue = 8, Cleric = 8, Barbarian = 12)  
# Calculating the hitpoints and combat stats:  
Hitpoints = hit_dice[class] + con_mod  
Attack = 2 + modifier(stats[["Strength"]])  
AC = 10 + modifier(stats[["Dexterity"]])
```

DGPs in Action

Given that we know the DGP, we can model (abstract) things very effectively, e.g.:

```
dnd_characters %>%  
  group_by(Class) %>%  
  summarise(mean_hitpoints = mean(Hitpoints),  
            sd_hitpoints = sd(Hitpoints)) %>%  
  ggplot(aes(x = Class, y = mean_hitpoints)) +  
  geom_point(size = 4) +  
  geom_errorbar(aes(ymin = mean_hitpoints - sd_hitpoints,  
                  ymax = mean_hitpoints + sd_hitpoints), width = 0) +  
  labs(x = "", y = "Mean Hitpoints (+- 1 Standard Deviation)") +  
  theme_minimal(base_size = 24)
```

DGPs in Action



This shows the CEF of hitpoints as a function of class!

Finishing Our Adventure

From Duel to Battle

Suppose our duelists bury the hatchet, recruit a Cleric to heal them, and go looking for goblins



From Duel to Battle

Our party of three finds four goblins. Each goblin has a fixed stat block (no randomness), and all goblins are identical by design:

```
# Recruit a Cleric to join the two duelists
cleric <- dnd_characters %>% filter(Class == "Cleric") %>% slice(1)
party <- bind_rows(barbarian, fighter, cleric)
party %>% select(Name, Class, Hitpoints, Attack, AC, Die)
```

	Name	Class	Hitpoints	Attack	AC	Die
1	Thinos	Barbarian	14	1	12	12
2	Thuldor	Fighter	12	2	12	8
3	Serulos	Cleric	9	4	11	6

```
# Goblins are identical by design: a fixed stat block
goblins <- data.frame(Name = paste("Goblin", 1:4), Hitpoints = 7,
                      Attack = 4, AC = 13, Die = 6)
goblins
```

	Name	Hitpoints	Attack	AC	Die
1	Goblin 1	7	4	13	6
2	Goblin 2	7	4	13	6
3	Goblin 3	7	4	13	6
4	Goblin 4	7	4	13	6

Simulating a Battle

Each round every standing combatant attacks a random standing enemy, and a coin flip again decides which side strikes first:

```
simulate_battle <- function(party, goblins) {  
  # One side's turn: everyone standing attacks a random standing enemy  
  take_turn <- function(attackers, hp_att, defenders, hp_def) {  
    for (i in which(hp_att > 0)) {  
      standing <- which(hp_def > 0)  
      if (length(standing) == 0) break  
      target <- standing[sample(length(standing), 1)]  
      if (sample(1:20, 1) + attackers$Attack[i] >= defenders$AC[target]) {  
        hp_def[target] <- hp_def[target] - sample(1:attackers$Die[i], 1)  
      }  
    }  
    hp_def  
  }  
  hp_party <- party$Hitpoints  
  hp_goblin <- goblins$Hitpoints  
  # Flip a coin: if the goblins win it, they get the first strike  
  if (sample(c(TRUE, FALSE), 1)) {  
    hp_party <- take_turn(goblins, hp_goblin, party, hp_party)  
  }  
  # Rounds of combat: the party acts, then the goblins answer  
  while (any(hp_party > 0) && any(hp_goblin > 0)) {  
    hp_goblin <- take_turn(party, hp_party, goblins, hp_goblin)  
    hp_party <- take_turn(goblins, hp_goblin, party, hp_party)  
  }  
  sum(hp_party > 0) # How many adventurers are left standing?  
}
```

One Battle Isn't Enough

Again, one battle is a single draw, so let's see the distribution of surviving adventurers over many:

```
set.seed(1)
simulate_battle(party, goblins)    # Survivors in one battle
```

```
[1] 0
```

One Battle Isn't Enough

Again, one battle is a single draw, so let's see the distribution of surviving adventurers over many:

```
set.seed(1)
simulate_battle(party, goblins)    # Survivors in one battle
```

```
[1] 0
```

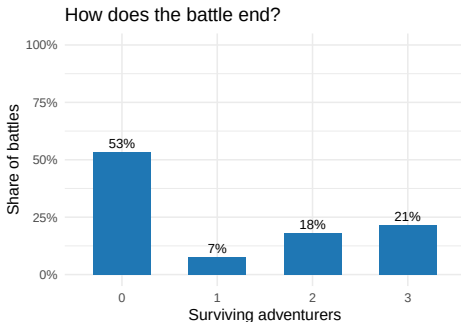
```
set.seed(1)
survivors <- replicate(10000, simulate_battle(party, goblins))
mean(survivors > 0)    # P(at least one survivor) = P(the party wins)
```

```
[1] 0.4674
```

Who Survives?

Four goblins turn out to be an even match for our party of three:

```
as.data.frame(prop.table(table(survivors))) %>%  
  ggplot(aes(x = survivors, y = Freq)) +  
  geom_col(fill = "#1f77b4", width = 0.6) +  
  geom_text(aes(label = scales::percent(Freq, accuracy = 1)),  
            vjust = -0.5, size = 7) +  
  labs(x = "Surviving adventurers", y = "Share of battles",  
       title = "How does the battle end?") +  
  scale_y_continuous(labels = scales::percent, limits = c(0, 1)) +  
  theme_minimal(base_size = 24)
```



How Many Goblins is Too Many?

Our party has had a good run. Now let's see where their luck would run out – how many goblins is too many? We control the DGP in our simulation, so we can change an input and simply re-run the simulation:

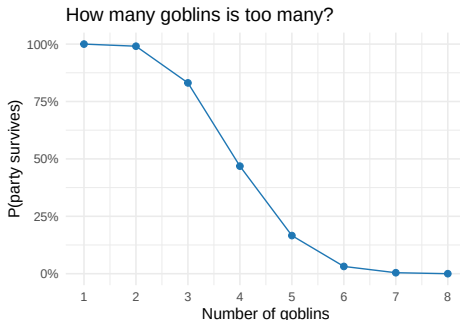
```
set.seed(1)
win_prob <- sapply(1:8, function(n) {
  goblins <- data.frame(Name = paste("Goblin", 1:n), Hitpoints = 7,
                        Attack = 4, AC = 13, Die = 6)
  mean(replicate(5000, simulate_battle(party, goblins)) > 0)
})
setNames(round(win_prob, 2), 1:8)
```

1	2	3	4	5	6	7	8
1.00	0.99	0.83	0.47	0.17	0.03	0.00	0.00

How Many Goblins is Too Many?

The result is the CEF of party survival as a function of the number of goblins. Two goblins are trivial but five are deadly:

```
data.frame(goblins = 1:8, win_prob) %>%  
  ggplot(aes(x = goblins, y = win_prob)) +  
  geom_line(color = "#1f77b4", linewidth = 1) +  
  geom_point(size = 4, color = "#1f77b4") +  
  labs(x = "Number of goblins", y = "P(party survives)",  
       title = "How many goblins is too many?") +  
  scale_x_continuous(breaks = 1:8) +  
  scale_y_continuous(labels = scales::percent, limits = c(0, 1)) +  
  theme_minimal(base_size = 24)
```



The End of One Adventure, The Start of Another

Today, we had a distinct advantage:

DGP $\longrightarrow$ data

We created the DGP, and simulated data on the basis of our assumptions.

The End of One Adventure, The Start of Another

Today, we had a distinct advantage:

$\text{DGP} \longrightarrow \text{data}$

We created the DGP, and simulated data on the basis of our assumptions.

Applied data science works the other way:

$\text{data} \longrightarrow \text{DGP?}$

The world hands us a dataset, and the process that made it is hidden from us.

The End of One Adventure, The Start of Another

Today, we had a distinct advantage:

$$\text{DGP} \longrightarrow \text{data}$$

We created the DGP, and simulated data on the basis of our assumptions.

Applied data science works the other way:

$$\text{data} \longrightarrow \text{DGP?}$$

The world hands us a dataset, and the process that made it is hidden from us.

All we ever get is the data. Our task is to learn things about the DGP from that data. That's tomorrow's adventure.